JWT 게시판 구현 보고서

이 양식은 본인이 구현한 결과를 설명하기 위한 문서입니다. 대괄호 안내를 실제 내용으로 바꾸고 작성 안내 문장은 제출 전에 지우세요. 화면 캡처와 표의 행은 필요에 따라 추가할 수 있습니다. 6~10쪽 내외를 권장하며 분량 자체는 평가하지 않습니다.

01 서비스 소개와 구현 범위

학번 **202446133** 이름: 유인상 제출일: 10/28

서비스 이름 **FixLog**

기술 조합과 버전 [React 19.3.0 (Vite 8.3.4) / Spring Boot 3.5.16 (Java 17 대상, 실행 JDK 21.0.1, Maven Wrapper 3.9.11) / JJWT 0.13.0 / SQLite (sqlite-jdbc 3.51.3.0) / Node.js 24.3.0 / macOS 26.6.2]

GitHub 저장소 URL [https://github.com/Luca388/my_board]

소스 코드 기준 [main / 구현 커밋 811fa8e / 최종 제출 기준 커밋 SHA는 LMS에 제출합니다.]

서비스의 대상 사용자와 사용 목적을 3~5문장으로 설명하세요. 이 기술 조합을 선택한 이유와 제공 템플릿에서 변경한 범위를 적으세요.

대상은 개발하다 오류를 만나는 학생 및 개발자이다, 개발하다 만나는 오류들을 기록하고 공유하는 용도가 목적이다 .

이 기술조합을 선택한 이유는 React는 가장 인기있는 프론트 엔드 라이브러리이기 때문에 풍부한 생태계가 장점이라는 것에서 선택하였고 Spring boot 는 이전에 사용해본 경험이 있어 기존 지식 복습 및 계층 분리구조를 사용해보기 위해 골랐다.

완료 여부는 완료·부분·미완료 중 하나로 기록하세요.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	완료	BoardController.list,get	로그인 없이 조회
로그인과 JWT 발급	완료	AuthService 생성자,issueToken / api.js login	HS256, TTL 90초
인증과 내 정보 조회	완료	AuthService.currentUserId/ BoardController.me	401 3종 구분
본인 글 작성·수정·삭제	완료	AuthService.currentUserId / api.js canEdit	타인 글 403
만료·로그아웃·오류 표시	완료	api.js request, main.jsx run	401 시 로그인 해제, 403 시 유지
DB 데이터 유지	완료	PostStore,DatabaseConfig	backend/data/board.sqlite

구조와 변경 내역

요청 흐름, 토큰의 이동, 변경한 파일의 역할을 연결해 설명하세요.

브라우저 → API 서버 → SQLite의 연결 구조를 간단히 그리거나 글로 설명하세요. 로그인 요청과 게시글 수정 요청의 흐름을 구분하고, 토큰이 어디에 저장되고 어느 요청에서 전송되는지 표시하세요.



변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유	작성출처
backend/.../AuthService.java 생성자	없음	jwt 설정 주입, 잘못된 키면 서버 시작 거부, 키를 32바이트로 변환	키 검사 요구	ai 작성, 그대로 반영
AuthService.issueToken	501 오류	hs256 토큰 발급	올바른 계정에만 토큰 발급	ai 작성, 그대로 반영
AuthService.currentUserId	항상 id 1 반환	토큰 검증 후 사용자 id 반환, 실패시 401	위조 만료 토큰 막고 실제 사용자 확인	ai 작성, 그대로 반영
AuthService.assertOwner	검사없음	작성자가 다르면 403	남의 글 수정 삭제를 서버에서 막기위해	ai작성, 그대로 반영
api.js login	미구현 오류	로그인후 토큰 사용자를 메모리에 저장	로그인상태를 유지하고 화면에 표시하기 위해	ai 작성, 그대로 반영
api.js authorizationHeader	빈 헤더	토큰이 있으면 Bearer헤더 추가	보호 api에 토큰을 보내기 위해	ai 작성, 그대로 반영
api.js request	401처리 없음	401이면 로그인 해제 , 오류 code 별 안내	만료 변조시 재로그인을 안내하기 위해	ai 작성, 그대로 반영
api.js canEdit	항상 true	본인 글인지 비교	본인 글에만 수정 삭제 버튼을 보이기 위해	ai 작성, 그대로 반영
main.jsx ,index.html,style.css	데모화면	FixLog문구, 오류 강조,비로그인 작성 제한 ,404 시 목록 갱신	서비스에 맞는 안내와 오류별 화면 처리를 위해	ai 작성, 그대로 반영
폴더 이름	frontend-react,backend-spring	frontend,backend	제출 구조를 맞추기 위해	본인 결정
.gitignore	없음	커밋 제외 규칙	키,db 를 커밋에서 빼기위해	ai 작성 , 그대로 반영

각 변경의 작성 출처도 구분하세요. 예: 제공 코드 유지, 본인 작성, AI 초안 후 본인 수정, 외부 예제 참고. 보조 파일을 추가했다면 함께 기록하세요.

로컬 실행 기록

실제 설치·빌드·실행 명령과 결과를 적고, 자세한 재현 방법은 저장소 README에서 연결하세요.

항목	내용
운영체제와 버전	macOS 26.6.2 / JDK 21.0.1 / Node.js 24.3.0
새 폴더 설치	저장소를 새 폴더에 clone → `.env`, `backend/data`, `node_modules`가 없는 상태(비밀 키·DB·의존성이 커밋되지 않았음을 확인) → `frontend`에서 `npm ci` 성공(취약점 0)
환경설정	README 명령으로 `backend/.env` 생성(64자리 키, `TOKEN_TTL=90`), `.env`는 git에 잡히지 않음. `source .env`로 서버에 전달
프론트 빌드	`npm run build` → `✓ built in 324ms`, `dist/` 생성
백엔드 실행	`source .env && ./mvnw spring-boot:run` → `Started BoardApplication`, `/api/health` → `{"status":"ok","stack":"spring"}`
SQLite 파일 경로	`backend/data/board.sqlite` — 새 폴더에서 서버를 처음 켤 때 폴더와 파일이 자동 생성됨
4173번 접속 및 API 연결 확인	`npm run preview` 빌드 화면에서 목록 요청 `GET /api/posts`와 로그인 `POST /api/auth/login`이 백엔드로 전달되어 "Alice님 로그인 중" 표시. 자동 검사 `check_api.py --mode final` 40개 통과
서버 재시작 후 데이터 유지	T15: 글(id 25) 작성 → 서버 종료·재실행 → 같은 내용 조회
증거 위치	`docs/evidence/` (t15-1, t15restart, t15-3 등), 재현 방법은 루트 `README.md`

핵심 코드 설명

네 부분을 각각 설명하세요. 전체 파일을 붙이지 말고 핵심 코드 5~20줄 정도를 발췌한 뒤 입력 → 처리 → 출력과 실패 조건을 자신의 말로 설명합니다. 성공·실패 시 HTTP 상태와 JSON code, 해당 분기를 만드는 코드도 연결해 적으세요. 줄 번호를 적는 경우 제출한 코드 기준인지 확인하세요.

3 1 로그인과 JWT 발급

파일 / 함수: [AuthService.java / AuthService() 생성자, issueToken()]

코드 발췌

```
if (secret == null || secret.isBlank()) throw new IllegalStateException(...);
if (secret.equals(EXAMPLE_SECRET))      throw new IllegalStateException(...);
if (!secret.matches("[0-9a-fA-F]{64}")) throw new IllegalStateException(...);
...
this.key = Keys.hmacShaKeyFor(HexFormat.of().parseHex(secret));
```

```
public String issueToken(Users.PublicUser user) {
  Instant now = Instant.now();
  return Jwts.builder()
      .subject(String.valueOf(user.id()))
      .issuer(issuer)
      .audience().add(audience).and()
      .issuedAt(Date.from(now))
      .expiration(Date.from(now.plusSeconds(ttl)))
      .signWith(key, Jwts.SIG.HS256)
      .compact();
}
```

동작과 응답 코드 설명

서버가 시작될때 AuthService 생성자가 환경변수로 받은 jwt_secret을 먼저 검사한다 , 값이 비어있거나 과제에서 제공한 예시 키이거나 64 자리 16진수 변수가 아니면 예외를 던져 서버가 시작되지 않게 했다 . 검사를 통과한 키는 16진수 문자열을 32바이트로 변환해 hs256서명 키를 만든다. 로그인 요청이 오면 BoardController.login()이 email,password 형식을 확인하고(오류발생 시 400 BAD_INPUT), User.verify() 로 비밀번호를 대조한다. 계정이 일치하지 않으면 401 INVALD_CREDENTIALS 를 반환하고 토큰을 발급하지 않는다 . 일치하면 issueToken() 이 sub 에 문자열 사용자 id, iss에 webservice-class,aud 에 board-api , exp 에 발급시간 + 90 초를 넣고 hs256으로 서명한다 . 응답은 200과 함께 accessToken,expiresIn,user[id,name,email} 이며 토큰과 응답에는 비밀번호나 해시를 넣지 않는다

3 2 브라우저 토큰 저장과 전송

파일 / 함수: [api.js / login() , authorizationHeaders(),request()]

코드 발췌

```
let session = { accessToken: "", user: null };
```

```
function authorizationHeaders() {
  return session.accessToken
    ? { Authorization: `Bearer ${session.accessToken}` }
    : {};
}
```

```
// request() 안
const data = response.status === 204 ? null : await response.json().catch(() =>
({}));
if (!response.ok) {
  if (response.status === 401) clearSession();
  ...
}
export async function login(email, password) {
  clearSession();
  const data = await request("/auth/login", { method: "POST", body:
JSON.stringify({ email, password }) });
  session = { accessToken: data.accessToken, user: data.user };
  return data.user;
}
```

동작과 응답 코드 설명

로그인 버튼을 누르면 login()이 먼저 기존 세션을 비운 뒤 POST /api/auth/login을 보낸다. 200 응답을 받으면 accessToken과 user를 모듈 변수 session에 저장하고, 화면에 "○○님 로그인 중"을 표시한다.

토큰은 localStorage나 sessionStorage에 저장하지 않고 메모리에만 두기 때문에 새로고침하면 로그인이 해제된다. 이후 모든 요청은 request()를 거치며, authorizationHeaders()가 토큰이 있을 때만 Authorization: Bearer <토큰> 헤더를 붙이고 토큰이 없으면 헤더 자체를 보내지 않는다.

응답이 204이면 본문이 없으므로 JSON을 파싱하지 않는다. 로그인이 400 BAD_INPUT이나 401 INVALID_CREDENTIALS로 실패하면 토큰을 저장하지 않고 오류 메시지만 보여 준다.

보호 API에서 401(TOKEN_REQUIRED, INVALID_TOKEN, TOKEN_EXPIRED)을 받으면 clearSession()으로 토큰과 사용자 정보를 지워 로그아웃 상태로 바꾸고 다시 로그인하라고 안내한다.

403 FORBIDDEN은 토큰이 유효한 상태의 권한 부족이므로 로그인을 유지한 채 안내만 한다. 로그아웃 버튼도 clearSession()을 호출해 메모리의 토큰과 화면 표시를 즉시 지운다.

서버 인증과 작성자 권한

아래 설명은 실제 제출한 파일과 함수의 동작을 근거로 작성하세요.

3 3 서버 JWT 검증

파일 / 함수: [AuthService.java / currentUserId()]

코드 발췌

```
String header = request.getHeader("Authorization");                          // 74
if (header == null || !header.startsWith("Bearer "))                          // 75
  throw new ApiError(401, "TOKEN_REQUIRED", "로그인이 필요합니다.");
Matcher m = BEARER.matcher(header);                                           // 77
if (!m.matches()) throw invalidToken();                                       // 78
Claims claims;
try {
  Jws<Claims> jws = Jwts.parser()
      .verifyWith(key)
      .requireIssuer(issuer)
      .requireAudience(audience)
      .build()
      .parseSignedClaims(m.group(1));                                         // 89
  if (!Jwts.SIG.HS256.getId().equals(jws.getHeader().getAlgorithm())) throw
invalidToken();  // 91
  claims = jws.getPayload();
} catch (ExpiredJwtException e) {                                             // 93
  throw new ApiError(401, "TOKEN_EXPIRED", "로그인 시간이 만료되었습니다. 다시 로그인하세요.");
} catch (JwtException | IllegalArgumentException e) {                         // 96
  throw invalidToken();   // 401 INVALID_TOKEN (112~114줄)
}
if (claims.getExpiration() == null) throw invalidToken();                     // 102
String sub = claims.getSubject();
if (sub == null || !USER_ID.matcher(sub).matches()) throw invalidToken();   // 106
long userId = Long.parseLong(sub);
if (users.find(userId) == null) throw invalidToken();                         // 108
return userId;
```

동작과 응답 코드 설명

currentUserId()는 /api/me와 글 등록·수정·삭제 요청에서 가장 먼저 호출된다. 먼저 Authorization 헤더가 있고 Bearer 로 시작하는지 확인하며, 헤더가 없거나 Bearer 로 시작하지 않으면 401 TOKEN_REQUIRED를 반환한다. Bearer 뒤의 값이 헤더.페이로드.서명 세 부분 형태가 아니면 토큰 자체가 잘못된 것으로 보고 401 INVALID_TOKEN을 반환한다.

다음으로 JJWT 파서에 서버 키를 지정해 서명을 검증하고, 발급자(iss)와 사용 대상(aud)이 설정값과 같은지, 만료되지 않았는지를 함께 확인한다. 토큰 내용을 읽기만 하는 decode가 아니라 서명을 다시 계산해 비교하므로, payload를 고친 토큰은 서명이 맞지 않아 거부된다. 이어서 헤더의 알고리즘이 HS256인지 확인하고, exp 클레임이 실제로 있는지 따로 검사한다. JJWT는 exp가 없으면 만료 검사를 건너뛰기 때문이다.

마지막으로 sub가 양의 정수 문자열인지, 그 id의 계정이 실제로 있는지 확인한 뒤 검증된 사용자 id만 반환한다. 정상 서명의 만료 토큰은 401 TOKEN_EXPIRED, 서명 불일치·클레임 오류·없는 사용자는 401 INVALID_TOKEN이며, 401 응답에는 Errors.java가 WWW-Authenticate: Bearer 헤더를 붙인다. 검증을 통과하면 /me는 200, 글 등록은 201처럼 원래 처리가 이어진다.

3 4 작성자 권한 검사

파일 / 함수: [AuthService.java / assertOwner()]

코드 발췌

```
public void assertOwner(long userId, long authorId) {
  if (userId != authorId)
    throw new ApiError(403, "FORBIDDEN", "본인이 작성한 글만 수정·삭제할 수 있습니다.");
}
// BoardController.update
long user = auth.currentUserId(req), pid = postId(id);   // 1) 인증
var p = existing(pid);                                   // 2) 글 존재 (404)
auth.assertOwner(user, p.authorId());                    // 3) 권한 (403)
var input = fields(body, Map.of("title", 100, "content", 5000)); // 4) 입력 (400)
```

동작과 응답 코드 설명

assertOwner()는 토큰으로 확인한 요청자의 id와 DB에 저장된 글의 작성자 id를 비교하고, 다르면 403 FORBIDDEN을 던진다. 두 값 모두 클라이언트가 조작할 수 없는 값이다. 요청자 id는 서명이 검증된 토큰에서, 작성자 id는 DB에서 가져오기 때문이다. update()와 delete()는 인증(currentUserId) → 글 존재 확인(existing) → 작성자 비교(assertOwner) → 입력 검증(fields) 순서로 처리한다.

그래서 토큰 문제는 401, 글이 없으면 404 NOT_FOUND, 남의 글이면 403이 나오고, 403으로 거부되면 DB는 바뀌지 않는다. 같은 이유로 Bob이 Alice의 글에 공백 제목을 보내면 입력 오류(400)보다 권한 검사가 먼저 실행되어 403이 된다. 401은 누구인지 확인할 수 없는 인증 실패이고, 403은 누구인지는 확인했지만 권한이 없는 경우라는 점이 다르다.

작성자 위조도 막는다. 글을 등록할 때 작성자는 요청 본문이 아니라 토큰에서 확인한 id로 정해지고, 본문에 authorId나 userId를 넣으면 400 BAD_INPUT으로 거부된다. 본인 글이면 수정은 200, 삭제는 204(본문 없음)로 처리된다. 프론트의 canEdit()은 버튼을 숨기는 표시용일 뿐이고 실제 권한은 서버의 assertOwner()가 검사한다.

테스트 결과

T01부터 T17까지 수행하고, 실제 응답과 증거의 위치를 기록하세요.

실행 환경: [macOS / JDK 21.0.1 / Node.js 24.3.0 / Chrome / Postman / 백엔드 source .env && ./mvnw spring-boot:run, 프론트 npm run build && npm run preview]

로컬 배포 확인: [새 폴더에서 설치한 절차 / 프론트 빌드 결과 / 백엔드 실행 명령 / 4173번 접속 및 API 연결 증거]

테스트 일시 및 코드 버전: [2026-10-27 ~ 28, 커밋 811fa8e 이후 코드]

토큰 TTL: [일반 테스트 값 : 90 / 만료 테스트 값 : 3]

05_테스트시나리오.md의 T01~T17 결과를 모두 기록하세요. PASS, FAIL, 미실행을 구분합니다. 예상 결과를 실제 결과로 복사하지 말고 직접 관찰한 HTTP 상태와 JSON code, 화면을 적으세요. 실제 결과 칸에는 예를 들어 403 / FORBIDDEN처럼 기록합니다. 정상 응답은 200 / 해당 없음처럼 적습니다.

ID	수행 절차와 사용자	예상 결과	실제 HTTP와 code	판정	증거
T01	비로그인으로 목록·상세 조회 (브라우저)	200	200/ 해당없음	PASS	t1.png
T02	Alice 로그인(브라우저)	200,jwt ,공개 사용자 정보,화면 표시	200/해당없음, expiresIn 90, user[1,Alice},"Alice 님 로그인중"	PASS	t2.png,t2jwt.png
T03	틀린 비밀번호, 없는 이메일 로그인 (postman)	401 INVALID_CREDENTIALS	401 / 화면 "이메일 또는 비밀번호가 올바르지 않습니다"	PASS	t3pwd.png,t3email.png
T04	토큰 없이 /me, 글 등록 (postman)	401 TOKEN_REQUIRED	401, WWW-Authenticate: Bearer	PASS	t4_me.png,t4write.png
T05	Alice 글 등록 후 /me (브라우저)	201,authorld 1 , /me Alice	201,Location /api/posts/14,authord 1 / /me Alice	PASS	t5write.png, t5authord.png,t5_me.png
T06	Alice 본인 글 수정, 삭제 (브라우저)	200 , 204	200, 204 , 목록에서 삭제됨	PASS	t6put.png,t6put_payload.png,t6delete.png
T07	Bob 토큰으로 Alice 글 삭제 (postman)	403 FORBIDDEN, 원문 유지	403 , FORBIDDEN, 이후 재조회 원문 유지	PASS	t7bobPut.png,t8review.png,t9check.png
T08	Bob 토큰으로 Alice 글 삭제 (postman)	403 , 글유지	403 , FORBIDDEN, 재조회, 200 원문 그대로	PASS	t8bobdelete.png, t8review.png
T09	서명 첫 글자 변경 토큰으로 /me, POST, PUT, DELETE (Postman)	401 INVALID_TOKEN, 변경 없음	401 / INVALID_TOKEN ×4, 재조회 원문 유지	PASS	t9.png, t9-2.png, t9put.png, t9-4.png, t9check.png

ID	수행 절차와 사용자	예상 결과	실제 HTTP와 code	판정	증거
T10	TTL 3초 토큰을 5초 뒤 /me, POST, PUT, DELETE (Postman) + 화면	401 TOKEN_EXPIRED, 화면 로그인 해제	expiresIn 3 / 401 TOKEN_EXPIRED ×4, 화면 "로그인 시간이 만료되었습니다" + 로그아웃 상태	PASS	t10-0.png ~ t10-4.png, t10expire.png
T11	Authorization 없이 /me (Postman)	401 TOKEN_REQUIRED	401 / TOKEN_REQUIRED	PASS	t11.png
T12	Bob 등록에 authorId:1, userId:1 (Postman)	400 BAD_INPUT, 위조 글 없음	400 / BAD_INPUT ×2, 목록에 Bob 글 없음	PASS	t12bobwrite.png, t12bobwrite2.png, t12notcreate.png
T13	빈 제목, 공백 본문, 제목 101자, 본문 5001자 (Postman)	400 BAD_INPUT	400 / BAD_INPUT ×4	PASS	t13emptytitle.png, t13emptycontent.png, t13longtitle.png, t13longcontent.png
T14	없는 글 99999 조회·수정·삭제, ID abc (Postman)	404, 잘못된 ID 400	404 / NOT_FOUND ×3, 400 / BAD_INPUT	PASS	t14nonesearch.png, t14noneput.png, t14nonedelete.png, t14wrongid.png
T15	글 작성 → 서버 종료·재실행 → 조회 (Postman + 터미널)	글·작성자·내용 유지	201(id 25) → 재시작 → 200, 내용·작성자·시각 동일	PASS	t15-1.png, t15restart.png, t15-3.png
T16	로그인 후 새로고침 (브라우저)	글 유지, 로그인 해제	해당 없음 (화면 테스트), 로그아웃 상태, 글 목록 유지	PASS	t16.png
T17	토큰 없는 수정·삭제, Bob 본인 작성·수정·삭제 (Postman)	무토큰 401, Bob 성공	401 TOKEN_REQUIRED ×2 / 201(authorId 2) → 200 → 204 → 404	PASS	t17-1.png ~ t17-6.png

HTTP 응답 확인 기록

예상 코드와 실제 코드를 대조하고, 201·204·401의 헤더와 본문도 확인하세요.

숫자는 Network나 API 클라이언트에서 직접 관찰한 값을 기록합니다. 본문에 있는 숫자만으로 판단하지 않습니다. 한 항목이 여러 요청을 포함하면 행을 추가하세요.

확인 항목	기대 응답	실제 HTTP와 code	증거 위치
정상 조회·로그인·수정	**200**	200	t1,t2,t6put,t17-4
새 글 등록	**201 및 Location**	201, Location: /api/posts/{id}	t5write, t17-3
본인 글 삭제	**204 및 빈 본문**	204, 본문 없음	t6delete, t17-5
입력 오류·추가 작성자 필드	**400 BAD_INPUT**	400 BAD_INPUT	t12, t13, t14wrongid
잘못된 계정	**401 INVALID_CREDENTIALS**	401 (화면 안내)	t3pwd, t3email
토큰 없음·헤더 형식 오류	**401 TOKEN_REQUIRED**	401 TOKEN_REQUIRED	t11, t17-1, t17-2
변조·클레임 오류	**401 INVALID_TOKEN**	401 INVALID_TOKEN	t9, t9-2, t9put, t9-4
정상 서명의 만료 토큰	**401 TOKEN_EXPIRED**	401 TOKEN_EXPIRED	t10-1 ~ t10-4
다른 작성자의 변경	**403 FORBIDDEN**	403 FORBIDDEN	t7bobPut, t8bobdelete
없는 글	**404 NOT_FOUND**	404 NOT_FOUND	t14nonesearch, t14noneput, t14nonedelete, t17-6

[보호 API 401의 WWW-Authenticate: Bearer 확인 결과와 증거]

보호 API가 401을 반환할 때 응답 헤더에 WWW-Authenticate: Bearer가 포함되는 것을 확인했다.

브라우저에서 토큰 없이 내 정보 조회와 글 등록을 요청했을 때 Network의 응답 헤더에 WWW-Authenticate: Bearer가 있었다(t4_me.png, t4write.png). Postman에서도 변조 토큰(T09), 만료 토큰(T10), Authorization 없는 요청(T11), 토큰 없는 수정·삭제(T17)의 401 응답마다 이 헤더를 검사하는 테스트가 통과했다(t9.png, t10-1.png, t11.png, t17-1.png 등).

이 헤더는 제공 코드 Errors.java가 상태 코드 401일 때 붙인다.

[401과 403에서 화면이 각각 어떻게 달라지는지 설명]

401은 토큰이 없거나 무효하다는 뜻이므로 화면이 로그아웃 상태로 바뀐다. 토큰 유효 시간을 3초로 설정하고 로그인한 뒤 만료된 상태에서 요청을 보내자 "로그인 시간이 만료되었습니다. 다시 로그인하세요." 안내가 나오고 로그인 표시가 "로그아웃 상태"로 바뀌었으며, 글 등록 영역에 로그인 안내가 나타나고 등록 버튼이 비활성화되었다(t10expire.png).

403은 로그인은 유효하지만 권한이 없는 경우라서 로그인 상태를 유지하고 "본인이 작성한 글만 수정·삭제할 수 있습니다." 안내만 표시하도록 했다(api.js의 request()는 401일 때만 clearSession()을 호출).
다만 화면에서는 남의 글에 수정·삭제 버튼이 표시되지 않아 403을 받는 상황이 생기지 않으므로, 403 응답은 Postman으로 Bob의 토큰을 사용해 직접 확인했다(t7bobPut.png, t8bobdelete.png).

실패 후 데이터와 복합 조건 검사

실패한 수정·삭제 뒤 원문 유지

Bob의 토큰으로 Alice 글(id 15)의 수정·삭제를 시도해 403을 받은 뒤 공개 조회하자 제목, 내용, 작성자 Alice, createdAt과 updatedAt이 그대로였다(t8review.png). 변조 토큰의 401 뒤에도 원문이 유지됐다(t9check.png). authorId·userId를 넣은 등록 요청은 400으로 거부되어 위조 글이 생성되지 않았다(t12notcreate.png).

같은 없는 글에 대한 무토큰·정상 토큰 요청

기존 Postman 검사에서 정상 토큰으로 없는 글(id 99999)을 수정하면 404 NOT_FOUND였다(t14noneput.png). 추가 검사에서는 같은 없는 글에 대해 토큰 유무를 바꾸어 실제 HTTP와 JSON code를 확인했다.

실제 요청 조건	실제 HTTP / code	판정
토큰 없이 없는 글 수정	401 / TOKEN_REQUIRED	PASS
정상 토큰으로 같은 없는 글 수정	404 / NOT_FOUND	PASS
Bob이 Alice 글에 공백 제목 제출	403 / FORBIDDEN	PASS
Alice가 본인 글에 공백 제목 제출	400 / BAD_INPUT	PASS

추가 검사는 2026-10-28 OpenAI Codex가 별도 테스트 clone의 API(127.0.0.1:18000)에서 실행했다. 없는 글은 ID 9007199254740991을 사용했고, 공백 제목 검사는 새 Alice 임시 글에 수행했다. 정상 JSON으로 PUT 요청을 보냈으며 토큰 원문과 비밀번호는 결과 파일에 남기지 않았다.

판단 순서와 데이터 유지

수정 요청은 인증 → 글 존재 → 작성자 권한 → 입력 검증 순서로 처리된다. 토큰이 없으면 글의 존재 여부와 관계없이 401이며, 정상 토큰으로 없는 글을 수정하면 404이다. 있는 Alice 글에 Bob이 공백 제목을 보내면 입력 오류보다 작성자 검사가 먼저여서 403, Alice가 본인 글에 같은 입력을 보내면 400이다.

네 요청 모두 기대값과 같았다. 거부된 요청 뒤 공개 재조회한 글은 내용·작성자·시각이 모두 같았고, 검사 후 임시 글을 삭제했다. 기존 수동 증거와 이번 보조 도구 검증 결과를 구분했다.

추가 검사 원본: docs/evidence/overlap-results.json
기존 수동 증거: t8review.png, t9check.png, t12notcreate.png, t14noneput.png, t17-1.png

테스트 증거

실제 캡처에서 요청·상태·본문 영역을 발췌했다. 전체 화면은 docs/evidence에 있으며 토큰 값은 가렸다.



그림 1. T02 로그인 응답: accessToken 가림, expiresIn 90, Alice. t2jwt.png



그림 2. T05 POST /api/posts: 201, Location, Bearer 요청 헤더. t5write.png

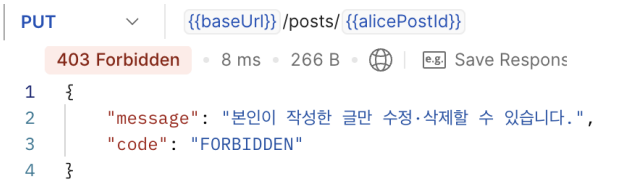


그림 3. T07 Bob의 Alice 글 수정(PUT): 403 FORBIDDEN. t7bobPut.png

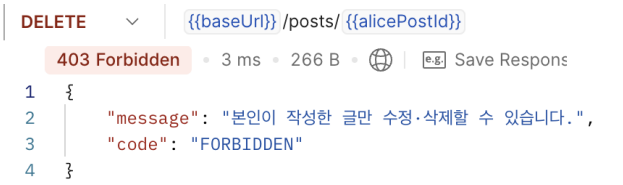


그림 4. T08 Bob의 Alice 글 삭제(DELETE): 403 FORBIDDEN. t8bobdelete.png



그림 5. T09 변조 토큰 /me: 401 INVALID_TOKEN. t9.png

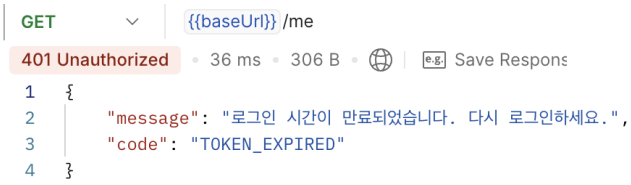


그림 6. T10 만료 토큰 /me: 401 TOKEN_EXPIRED. t10-1.png

화면 처리: t10expire.png(만료 안내·로그아웃). 실행·재시작: 4쪽 및 t15-1.png, t15restart.png, t15-3.png. 자동 API 검사: python3 tools/check_api.py --mode final, 40개 통과. 새 clone의 추가 검사에서는 --base http://127.0.0.1:14173으로 프론트 프록시를 경유했다.

AI 활용과 참고 자료

AI 사용 여부: 사용. 코드 생성·디버깅과 보고서 검수·다듬기에 활용한 내용을 구분했다.

도구·모델	사용 목적	반영 위치	반영 방식
Claude Code Claude Opus 5.5	JWT·프론트 코드 생성	AuthService.java, api.js, main.jsx, index.html, style.css	생성 코드 그대로 반영
Claude Code 동일 모델	시작 오류 분석·Postman 컬렉션 생성	로컬 .env, 실행 명령, tools/postman	참고 및 그대로 반영; 원래 수동 캡처는 본인 수행
Codex GPT-6.1 Sol / High	보고서 검수 및 다듬기	보고서	검수 의견을 참고해 문장과 구성을 다듬음

대표 사례 1 서버 시작 오류

도구·날짜: Claude Code / 2026-10-27

요청: ./mvnw spring-boot:run 실행 시 BeanCreationException이 발생한 원인과 해결 방법을 질문했다.

제안: JWT_SECRET이 없으면 application.properties의 예시 키가 사용되어 AuthService 생성자가 시작을 거부한다. 새 키를 만든 뒤 Spring이 .env를 자동으로 읽지 않으므로 source .env와 함께 실행해야 한다고 설명했다.

반영 위치: 로컬 backend/.env와 실행 명령. 실제 키는 커밋하지 않았다.

확인·수정: 단순히 .env 파일을 만드는 것만으로 해결되지 않는다는 점을 확인하고 source .env && ./mvnw spring-boot:run으로 실행 방법을 바꿨다.

검증: 서버가 Started BoardApplication 로그를 출력하고 /api/health가 정상 응답하며 로그인 요청이 200을 반환하는지 확인했다. 추가 clone 검사에서도 새 키로 동일한 시작 절차를 확인했다.

대표사례 2

도구 / 날짜: Claude Code (Claude Opus 5.5) / 2026-10-27

요청한 내용: 과제 API 명세에 맞게 AuthService의 토큰 발급(BE-01), 토큰 검증(BE-02), 작성자 권한 검사(BE-03)를 구현해 달라고 요청함. 먼저 구현 순서 설명을 받은 뒤 코드 작성을 요청함.

AI가 제안한 내용: 생성자에서 키를 검사하고 32바이트로 변환, HS256으로 sub·iss·aud·exp를 넣어 발급, 헤더 형식 → 서명·iss·aud·만료 → 알고리즘 → exp 존재 → sub와 계정 순서로 검증, 작성자가 다르면 403

실제 반영 위치: backend/src/main/java/edu/classroom/board/AuthService.java (BE-01용 import 7줄은 본인 작성)

내가 확인하고 수정한 내용: 코드는 수정 없이 반영했다. 대신 각 함수의 동작과 키 검사를 생성자에서 하는 이유, JJWT가 exp가 없으면 만료 검사를 건너뛰어 exp 존재를 따로 확인해야 하는 이유를 질문해 이해한 뒤 반영했다.

검증 방법과 결과: Postman으로 T02(로그인 200), T07·T08(Bob의 수정·삭제 403), T09(변조 토큰 401 INVALID_TOKEN), T10(만료 토큰 401 TOKEN_EXPIRED)을 직접 수행해 모두 기대 결과와 같음을 확인했다.

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
과제 공지 (`docs/etc/01_과제공지`)	전체	필수 기능, 제출물, AI 활용 기록 기준
공통 API 명세 (`docs/etc/02_API명세.md`)	서버 응답, 4절	HTTP 상태·JSON code·헤더 계약
구현 체크리스트 (`docs/etc/03_구현체크리스트.md`)	구현 확인	BE-01~03, FE-01~04 완료 기준
테스트 시나리오 (`docs/etc/05_테스트시나리오.md`)	4절	T01~T17 절차와 기대 결과
로컬 실행 안내 (`docs/etc/07_로컬실행안내.md`), 제출용 README 양식	실행·README	실행 명령, README 구성
과제 시작 템플릿 (`frontend-react`, `backend-spring`)	제공 코드 전체	라우트·DB·계정·화면 기본 구조 (출처: 과제 제공 코드)

웹서비스설계 06

이해도 확인과 회고

각 질문에 본인 코드의 동작을 근거로 답하세요.

1. 요청 본문의 authorId를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

요청 본문의 authorld 를 믿으면 다른사람의 id 를 넣어 그사람 이름으로 글을 쓸수있고 내 코드는 작성자를 본문이 아니라 토큰에서 검증한 사용자 id 로 저장하고 본문에 authorld,user id 가 있으면 400 bad input 으로 거부한다

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

버튼은 화면에서만 숨겨질뿐 postman 등으로 api 를 직접 호출할수있다 , 서버의 AuthService.assertOwner() 가 토큰의 사용자 id 와 작성자 id 를 비교해 다르면 403 forbidden 으로 거부한다

3. JWT decode와 verify의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

decode 는 base64로 된 내용을 읽기만해서 누구나 할수있고 이조도 막지 못한다 verify 는 서버 키로 서명을 다시 계산해 비교하기때문에 내용이 바뀌면 실패한다 , currentUserId() 는 서명 iss,aud 만료 알고리즘 ,exp 존재 sub 와 실제 계정을 확인한다

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

로그아웃과 새로고침은 브라우저 메모리 토큰만 지우고 게시글은 db 에 그대로 남는다 , 만료는 토큰의 exp 를 기준으로 서버가 판단하고 만료시각이 지나면 정상 서명의 토큰이어도 token_expired 를 받는다 이미 복사한 토큰은 토큰 만료 전까지 유효하다

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

BE-01을 구현한 뒤 ./mvnw spring-boot:run을 실행하자 BeanCreationException과 함께 서버가 시작되지 않았다.
로그의 Caused by 줄을 확인하니 AuthService 생성자에서 "예시 키는 사용할 수 없습니다" 예외가 발생하고 있었다. 원인은 JWT_SECRET 환경변수가 설정되지 않아 application.properties의 기본값인 예시 키가 들어갔고, 내가 넣은 키 검사가 이를 거부한 것이었다. 새 키를 만들어 backend/.env에 저장했지만, 같은 명령으로 다시 실행해도 같은 오류가 났다. Spring은 .env 파일을 자동으로 읽지 않고, 환경변수는 source .env를 실행한 터미널에서 띄운 프로세스에만 전달된다는 것을 확인했다. 그래서 source .env && ./mvnw spring-boot:run으로 실행 방법을 바꿨고, 서버가 정상적으로 시작되어 로그인이 200으로 응답하는 것을 확인했다

현재 남은 문제와 다음에 개선할 점: 없음

제출 확인: [ ] 실제 코드와 설명 일치 [ ] 테스트 직접 수행 [ ] AI 활용 및 출처 기록 [ ] 비밀 키와 토큰 제거

GitHub 확인: [ ] 루트 README 작성 [ ] 보고서 PDF와 증거 업로드 [ ] README 문서 링크 확인 [ ] 교수자 접근 가능 [ ] 저장소 URL과 커밋 SHA 제출